

Java Reserved Words

Created by ApnaAlgo

Visit us on YouTube: <https://www.youtube.com/@ApnaAlgo>

What Are Reserved Words?

Reserved words in Java are special commands that tell the computer what to do. They're like secret codes in a video game—you

can't use them for anything else, like naming your character or items. These words have specific jobs in Java programs.

Total Reserved Words in Java: 53

(No new reserved words have been added up to Java 21, but some keywords have cool new features!)

Types of Reserved Words

Java's reserved words come in two main types:

1. **Keywords:** Words that perform tasks, like making decisions or setting up data.
 2. **Literals:** Words that represent specific values, like `true`, `false`, or `null`.
-

Reserved Literals (3)

Literals are like labels for fixed values in Java, similar to saying "yes," "no," or "empty" in real life.

- **true:** Represents "yes" or "on" for boolean values.

```
boolean isGameOn = true;
```

- **false:** Represents "no" or "off" for boolean values.

```
boolean isGameOver = false;
```

- **null:** Represents "nothing" or "empty" for object references.

```
String playerName = null;
```

Real-World Example

Imagine a light switch:

- **true**: The light is on. **false**:
- The light is off. **null**: There's
- no light bulb installed!

Practical Example

```
boolean hasHomework = false; // false means no homework
String favoriteToy = null;    // null means no toy chosen
System.out.println("Homework today? " + hasHomework);
```

Output:

```
Homework today? false
```

Keywords (50)

Keywords are the superheroes of Java! They handle tasks like storing data, making decisions, or fixing errors. Out of the 53 reserved words, 50 are keywords, and 2 are unused.

1. Keywords for Data Types (8)

These keywords decide what kind of information a variable can hold, like numbers, letters, or true/false values.

- **byte**: For tiny numbers (e.g., `byte lives = 3;`).
- **short**: For small numbers (e.g., `short points = 500;`).
- **int**: For regular numbers (e.g., `int score = 100;`).
- **long**: For huge numbers (e.g., `long distance = 1000000L;`). **float**:
- For decimals (e.g., `float price = 9.99f;`). **double**: For precise
- decimals (e.g., `double temperature = 36.6;`). **boolean**: For
- true/false values (e.g., `boolean isWinner = true;`). **char**: For
- single characters (e.g., `char grade = 'A';`).

Real-World Example

In a soccer game app:

- Use **int** to count goals.
- Use **float** for average goals per match.
- Use **boolean** to check if the game is over.

Practical Example

```
int goalsScored = 2;           // Number of goals in a match
double averageGoals = 1.5;     // Average goals per match
boolean isMatchOver = true;    // Is the match finished?
System.out.println("Goals: " + goalsScored + ", Avg: " + averageGoals + ", Over? " + isMatchOver);
```

Output:

```
Goals: 2, Avg: 1.5, Over? true
```

2. Keywords for Flow Control (11)

These keywords steer your program, like a GPS telling you when to turn, stop, or keep going.

- **if:** Checks a condition (e.g., `if (score > 50) { ... }`). **else:** Offers an alternative (e.g., `else { ... }`). **switch:** Chooses from multiple options (e.g., `switch (level) { ... }`).
- **case:** Defines an option in a switch (e.g., `case 1: ...`).
- **default:** The fallback option (e.g., `default: ...`).
- **while:** Repeats while a condition is true (e.g., `while (lives > 0) { ... }`).
- **do:** Runs code at least once (e.g., `do { ... } while (condition);`).
- **for:** Loops a set number of times (e.g., `for (int i = 0; i < 10; i++) { ... }`).
- **break:** Stops a loop or switch (e.g., `break;`).
- **continue:** Skips to the next loop cycle (e.g., `continue;`).
- **return:** Sends back a result (e.g., `return score;`).

Real-World Example

At a pizza shop:

- Use **if** to check if a customer wants extra cheese.
- Use **switch** to select pizza size (small, medium, large).
- Use **for** to count how many pizzas to bake.

Practical Example

```
int pizzaSize = 2; switch
(pizzaSize) {      case 1:
    System.out.println("Small pizza ordered!");
break;      case 2:
    System.out.println("Medium pizza ordered!");
break;      default:
    System.out.println("Large pizza by default!");
}
```

Output:

```
Medium pizza ordered!
```

3. Keywords for Modifiers (11)

Modifiers are like settings that change how classes, methods, or variables behave.

- **public**: Anyone can access it (e.g., `public int score;`). **private**: Only the class can access it (e.g., `private String password;`). **protected**: The class and its subclasses can access it (e.g., `protected double balance;`).
- **static**: Belongs to the class, not objects (e.g., `static int totalPlayers;`).
- **final**: Cannot be changed (e.g., `final int MAX_LEVEL = 10;`).
- **abstract**: For incomplete classes/methods (e.g., `abstract class Vehicle { ... }`).
- **synchronized**: Controls access in multithreaded programs (e.g., `synchronized void update() { ... }`).
- **volatile**: Ensures variable consistency across threads (e.g., `volatile boolean isRunning;`).
- **transient**: Skips during serialization (e.g., `transient int temp;`). **native**: Links to non-Java code (e.g., `native void connect();`). **strictfp**: Ensures consistent floating-point calculations (since Java 1.2).

Real-World Example

In a school app:

- **public** makes student names visible to everyone.
- **private** keeps their grades hidden. **final** sets a maximum number of students per class.

Practical Example

```
public class School {    private String studentName;    public static int totalStudents = 0;    public final int MAX_STUDENTS = 30;    public School(String name) {        studentName = name;        totalStudents++;    }    public String getName() {        return studentName;    }    }
```

4. Keywords for Exception Handling (6)

These keywords act like a safety net, catching errors to keep your program running smoothly.

- **try**: Tests risky code (e.g., `try { ... }`). **catch**: Handles errors (e.g., `catch (Exception e) { ... }`). **finally**: Runs no matter what (e.g., `finally { ... }`).
- **throw**: Creates an error (e.g., `throw new Exception("Error!");`).
- **throws**: Warns about possible errors (e.g., `void risky() throws Exception { ... }`).
- **assert**: Checks assumptions during debugging (since Java 1.4, e.g., `assert score >= 0;`).

Real-World Example

When ordering food online:

- **try** checks if the payment goes through.
- **catch** handles a failed payment.
- **finally** sends a confirmation email.

Practical Example

```
try {      int itemsInCart = 0;      int totalItems = 10 /
itemsInCart; // Error: dividing by zero!
} catch (ArithmeticException e) {
    System.out.println("Cart is empty! Add some items.");
} finally {
    System.out.println("Thanks for shopping!"); }
```

Output:

```
Cart is empty! Add some items.
Thanks for shopping!
```

5. Keywords for Classes and Objects (7)

These keywords help you build and organize your program's structure, like blueprints for a building.

- **class**: Creates a class (e.g., `class Car { ... }`).
- **interface**: Defines a contract (e.g., `interface Drivable { ... }`).
- **extends**: Inherits a class (e.g., `class SportsCar extends Car { ... }`).
- **implements**: Uses an interface (e.g., `class Truck implements Drivable { ... }`).
- **package**: Groups classes (e.g., `package com.okaytechie;`).
- **import**: Brings in external code (e.g., `import java.util.Scanner;`).
- **enum**: Groups constants (since Java 1.5, e.g., `enum Day { MONDAY, TUESDAY }`).

Real-World Example

In a gaming app:

- **class** defines a "Player." **extends** creates a
- "SuperPlayer" with extra powers. **enum** lists game
- modes (easy, hard).

Practical Example

```
enum GameMode {
    EASY, MEDIUM, HARD
} class Player
{
    GameMode mode;
    Player(GameMode m) {
        mode = m;
    } void
    showMode() {
        System.out.println("Playing in " + mode + " mode!");
    }
}
```

6. Keywords Related to Objects (4)

These keywords help you work with objects, like tools for building and checking things.

- **new**: Creates objects (e.g., `String name = new String("Alex");`). **instanceof**:
- Checks an object's type (e.g., `if (obj instanceof String) { ... }`).
- **super**: Calls the parent class (e.g., `super.start();`). **this**:
- Refers to the current object (e.g., `this.name = name;`).

Real-World Example

In a car racing game:

- **new** creates a new car. **instanceof**
- checks if it's a sports car. **this** refers
- to the car's own speed.

Practical Example

```
class Vehicle {
    String type = "Generic Vehicle";
} class Car extends Vehicle {
    String type = "Sports Car";
    void showTypes() {
        System.out.println("This car: " + this.type);
        System.out.println("Parent: " + super.type);
    }
}
```

7. Void Return Type Keyword (1)

- **void**: Says a method doesn't return anything (e.g., `void playSound() { ... }`).

Real-World Example

In a music app, a **void** method plays a song without returning any data, just making sound happen.

Practical Example

```
void playMusic() {  
    System.out.println("Playing your favorite song!");  
}
```

Unused Keywords (2)

These keywords are reserved but not used in Java. Trying to use them will cause an error!

- **goto**: Banned because it made code messy in older languages.
- **const**: Replaced by `final`.

Real-World Example

Think of a "Do Not Enter" sign. **goto** and **const** are like signs Java put up to avoid confusion, so you can't use them.

Practical Example (Won't Work!)

```
goto end; // Error: goto is unused  
const MAX = 100; // Error: use final
```

What's New in Java 21?

Java 21 (released September 2023) is the latest long-term support (LTS) version. No new reserved words were added, but keywords like `switch` and `record` (introduced in Java 14) have awesome upgrades:

- **Switch Expressions**: `switch` can now return values directly, like picking an answer from a list.
- **Records**: A shortcut to create simple classes (uses the `record` keyword).

Real-World Example

In a weather app:

- **switch** picks a message based on the day (e.g., "Sunny" or "Rainy").
- **record** stores weather data neatly.

Practical Example (Switch Expression)

```
String weather = "SUNNY";
String forecast = switch (weather) {    case "SUNNY" -> "Wear
sunglasses!";    case "RAINY" -> "Bring an umbrella!";
default -> "Check the weather!";
};
System.out.println(forecast);
```

Output:

```
Wear sunglasses!
```

Enum in Action

The **enum** keyword (since Java 1.5) is great for grouping related constants, like a menu of choices.

Real-World Example

In a restaurant app, **enum** lists food categories (e.g., Pizza, Burger, Dessert) to make ordering easier.

Practical Example

```
enum Food {
    PIZZA, BURGER, DESSERT
} public class Restaurant {    public
static void main(String[] args) {
    Food order = Food.PIZZA;
    System.out.println("You ordered: " + order);
}
}
```

Output:

```
You ordered: PIZZA
```

Key Takeaways

- Java's **53 reserved words** (50 keywords + 3 literals) are the building blocks of every program.
- All keywords are **lowercase** and can't be used for variable names.
- **Java 21** adds cool features like switch expressions and records, but no new reserved words.
- No **delete** keyword in Java—the **garbage collector** cleans up unused objects automatically!

Fun Fact: Unlike C, Java requires a return type for methods. Use `void` if nothing is returned!

Questions and Answers

Questions

1. Which list contains only reserved words?

- a. new, delete
- b. goto, constant
- c. break, continue, return, exit
- d. final, finally, finalize
- e. throw, throws, thrown
- f. notify, notifyAll
- g. implements, extends, imports
- h. sizeof, instanceof
- i. byte, shortInt
- j. instanceof, strictFp
- k. None of the above

2. Which are Java reserved words?

- a. public
- b. static
- c. void
- d. main
- e. String
- f. args

Answers

1. **k. None of the above**

Explanation:

- o delete, constant, exit, finalize, thrown, notify, notifyAll, imports, sizeof, shortInt, instanceof, and strictFp are not Java reserved words. Some are methods, incorrect spellings, or non-existent in Java.

2. **a. public, b. static, c. void**

Explanation:

- o public is a modifier keyword.
- o static is a modifier keyword.
- o void is a return type keyword.
- o main is a method name, not a keyword.
- o String is a class, not a keyword.
- o args is a variable name, not a keyword.

Visit us on YouTube: <https://www.youtube.com/@ApnaAlgo>

